

Influence de la granularité des microservices et de leur déploiement conteneurisé sur les performances d'une application distribuée simulée

Alban GODIER¹

¹FISE Science du numérique, CESI École d'ingénieur, Saint-Nazaire, 44600, France

ABSTRACT

La granularité des microservices et le mode de déploiement conteneurisé sont deux paramètres architecturaux majeurs dont l'impact combiné sur les performances reste peu étudié en simulation contrôlée. Cette étude évalue six configurations issues du croisement de trois niveaux de granularité et trois modes d'isolation conteneurisée, testées sur dix applications distribuées générées aléatoirement sous sept niveaux de charge. Les résultats montrent que l'isolation conteneurisée est le facteur déterminant à forte charge, réduisant la latence et les échecs au prix d'une consommation CPU élevée, et révèlent un découplage inattendu entre utilisation CPU et latence selon le mode de déploiement.

Mots clés

Microservices, Granularité, Performance, Déploiement conteneurisé, Application distribuée

Introduction

Lors de la conception d'une architecture d'une application, il est souvent nécessaire de faire le choix entre une architecture monolithique ou une architecture en microservices. Cette dernière est largement adoptée pour concevoir des systèmes distribués évolutifs et maintenables. Elle s'appuie sur le principe de découpage d'une application en services autonomes, chacun implémentant une fonctionnalité spécifique et communiquant avec les autres via des interfaces bien définies. On nomme ces composants logiciels des microservices et leur niveau de granularité, c'est-à-dire la taille et la portée fonctionnelle de chaque service, constitue une décision importante qui influence directement les performances du système. Une granularité fine favorise l'isolation des pannes et la scalabilité, mais augmente la charge réseau et la complexité. À l'inverse, une granularité plus grossière limite les communications réseau tout en réduisant l'indépendance des composants¹. Cette étude vise à analyser expérimentalement, au travers de simulations, l'impact de différents niveaux de granularité sur les performances, afin d'identifier les compromis architecturaux les plus pertinents dans un contexte contrôlé.

Le choix de la simulation n'est pas anodin, car il permet de contrôler précisément les variables d'intérêt (granularité, charge de travail, etc.) et d'obtenir des données de performance dans un environnement isolé. Cependant, il est important de noter que les résultats obtenus à partir de simulations peuvent ne pas être entièrement représentatifs des conditions réelles d'une application distribuée en production. Les simulations peuvent simplifier certains aspects du système, tels que les interactions complexes entre les microservices, les variations de charge et les comportements imprévus. Par conséquent, les conclusions tirées de cette étude doivent être interprétées avec prudence et complétées par des études empiriques sur des applications réelles pour valider les résultats.

Les simulations ont été réalisées en utilisant des outils de conteneurisation tels que Docker, qui permettent de créer des environnements isolés pour chaque microservice. Nous avons utilisé des scripts Python pour simuler les microservices et générer des charges de travail variées. Les performances ont également été mesurées à l'aide de scripts Python permettant un monitoring en temps réel des ressources utilisées par chaque microservice.

Analyse documentaire

La granularité des microservices est un sujet largement documenté dans la littérature, avec des études qui mettent en évidence les avantages et les inconvénients de différentes approches. Par exemple, certains auteurs soutiennent que une granularité fine permet une meilleure isolation des pannes et une scalabilité accrue, tandis que d'autres soulignent que cela peut entraîner une complexité accrue et des coûts de communication plus élevés². D'autres études ont également exploré

les métriques utilisées pour évaluer la granularité des microservices, telles que le couplage, la cohésion et la complexité du code³. Enfin, des recherches récentes ont examiné l'impact de la granularité sur la consommation d'énergie et les performances, mettant en évidence des compromis importants à considérer lors de la conception d'une architecture en microservices⁴.

Afin de constituer cette analyse documentaire, cinq articles ont été sélectionnés pour leur proximité avec la thématique étudiée. Leur analyse a été structurée grâce à la méthode QALMRI qui permet d'extraire les éléments clés de chaque étude, tels que les questions de recherche, les hypothèses, les méthodes utilisées, les résultats obtenus et les conclusions. Les résultats de cette analyse documentaire ont permis de sélectionner un article de référence couvrant principalement la définition de la granularité des microservices et les métriques associées³, ainsi que quatre articles traitant empiriquement l'impact de la granularité sur les performances^{1,2,4,5}.

La première tendance identifiée dans la littérature est le coût de la communication entraîné par une granularité fine. En effet, plus les microservices sont petits et nombreux, plus ils doivent communiquer entre eux pour réaliser leurs tâches, ce qui peut entraîner une augmentation de la latence et une diminution des performances globales du système². Toutefois, il faut noter que cette augmentation de la latence est faible dans les systèmes de petite taille, mais devient significative à mesure que la taille du système augmente⁴.

Cette tendance est confirmée par une étude plus récente qui a évalué l'impact de la granularité sur les performances d'une application distribuée⁵. Les résultats ont montré que le découpage en microservices peut entraîner une augmentation significative de la latence, en particulier lorsque les microservices sont déployés sur des machines virtuelles distinctes. Cette tendance semble confirmée par l'expérience menée par Camila Medeiros Rêgo, Ricardo César Mendonça Filho et Nabor C. Mendonça¹, qui a montré que la distribution des services sur plusieurs nœuds peut améliorer la scalabilité, mais au prix d'une charge de communication plus importante.

Un deuxième axe semble se dessiner en ce qui concerne la consommation des ressources du système dans son ensemble. En effet, une granularité fine peut entraîner une utilisation plus importante des ressources, notamment en termes de mémoire et de CPU, en raison de la multiplication des processus et des communications entre les microservices⁴. Cependant, il est important de noter que cette augmentation de l'utilisation des ressources peut être compensée par les avantages en termes de scalabilité et d'isolation des pannes offerts par une granularité fine.

L'ensemble de ces sources semble converger vers une amélioration de la modularité, de l'isolement et de la scalabilité en adoptant une granularité fine, mais au prix d'une augmentation de la latence et de l'utilisation des ressources. Il est donc crucial de trouver un équilibre entre ces différents facteurs pour concevoir une architecture en microservices efficace et performante. Il est également important de noter que les résultats obtenus dans ces études montrent que la relation entre granularité et performance n'est pas linéaire car elle dépend de l'architecture utilisée, de la charge de travail, du contexte d'exécution ou encore de la méthode de communication inter-service choisie.

Pour conclure cette analyse, il semble y avoir deux limites récurrentes dans les études traitant de l'impact de la granularité sur les performances. Tout d'abord, la plupart des études se concentrent sur des systèmes de petite taille, ce qui limite la généralisation des résultats à des systèmes plus complexes et de grande taille. Ensuite, peu de travaux se concentrent explicitement sur la granularité, le déploiement conteneurisé contrôlé et les performances dans un cadre de simulation. Cette étude vise à combler ces lacunes en évaluant ces différentes métriques dans un environnement simulé et maîtrisé afin d'évaluer comment plusieurs niveaux de granularité et de déploiement conteneurisé influencent les performances d'une application distribuée.

Méthodes et données

Conception expérimentale

L'objectif de cette étude est d'évaluer l'impact de la granularité des microservices et de leur déploiement conteneurisé sur les performances d'une application distribuée simulée. Pour ce faire, nous avons conçu une expérience contrôlée articulée autour de trois niveaux de granularité (fin, moyen, grossier) et trois niveaux de conteneurisation (un conteneur unique, plusieurs conteneurs regroupant entre 1 et 4 services, un conteneur par service).

Ces deux facteurs nous ont permis de formuler les hypothèses suivantes: (H1) une granularité plus fine entraînera une augmentation de la latence en raison de la multiplication des communications inter-services; (H2) le déploiement multi-conteneurs augmentera la latence en raison de la surcharge liée à la virtualisation; (H3) à l'inverse, une granularité grossière entraînera une dégradation des performances sous forte charge, en raison de la concentration des requêtes sur un seul service.

Environnement de simulation

L'environnement de simulation est basé sur Docker (version 29.4.0) et Docker Compose (version 5.1.2). Chaque service est simulé par un script Python (version 3.14.4, en environnement virtuel) démarrant un serveur HTTP simple multithreadé, ce qui permet d'éviter les blocages liés à une file d'attente trop courte et de se rapprocher d'un fonctionnement réaliste. Les microservices effectuent une tâche de calcul représentative d'une charge de travail réelle: 20 000 itérations d'une division

euclidienne. Les conteneurs sont configurés avec une limite d'utilisation CPU afin de simuler les contraintes d'un système distribué réel et d'accentuer les différences de performances entre configurations.

Scénarios et variables

Pour la suite, nous définissons trois constituants d'un système distribué simulé: le *microservice*, représentant une tâche simple; le *service*, regroupant un ensemble de microservices; et le *conteneur*, regroupant un ensemble de services et symbolisant l'isolement matériel. Ces constituants donnent lieu aux six configurations suivantes:

1. Granularité fine + un conteneur unique (application monolithique)
2. Granularité moyenne + un conteneur unique
3. Granularité grossière + un conteneur unique
4. Granularité moyenne + plusieurs conteneurs (1 à 4 services par conteneur)
5. Granularité grossière + plusieurs conteneurs (1 à 4 services par conteneur)
6. Granularité grossière + un conteneur par service

L'expérimentation débute par la génération aléatoire de 10 applications distribuées afin d'assurer la généralisabilité des résultats. Chacune est ensuite déclinée selon les 6 configurations, produisant 60 configurations au total. Celles-ci sont soumises à des charges de travail croissantes: 1, 5, 10, 50, 100, 500 et 1 000 requêtes simultanées, soit 480 expériences au total (60 configurations × 8 niveaux de charge). Selon les hypothèses formulées, les configurations 1 et 6 devraient présenter les latences les plus élevées, et la configuration 3 devrait se dégrader plus rapidement sous forte charge.

Mesures et métriques

Trois métriques principales sont collectées pour chaque expérience: la latence moyenne (en millisecondes), le nombre d'échecs de requêtes, et l'utilisation CPU moyenne par conteneur (en pourcentage). Les ressources des conteneurs sont mesurées via la commande `docker stats`. Les temps de réponse et les échecs sont enregistrés par des scripts Python qui émettent des requêtes HTTP vers les services et consignent les réponses.

Procédure d'exécution et collecte des données

Les expériences sont orchestrées par des scripts Python utilisant la bibliothèque `subprocess` pour piloter les commandes Docker. Les résultats sont stockés dans des fichiers CSV structurés par configuration et niveau de charge, avec une ligne par expérience contenant la latence moyenne, le taux d'échec et l'utilisation CPU moyenne. Ces données brutes sont ensuite agrégées pour l'analyse statistique. L'agrégation consiste à calculer les moyennes et les écarts-types pour chaque configuration et niveau de charge, afin de faciliter la comparaison entre les différentes configurations. Nous avons alors pu tracer trois histogrammes représentant respectivement les trois métriques en fonction de la configuration et du niveau de charge.

Résultats

Cette partie se concentre sur l'analyse des histogrammes ci-contre (Figures 2, 3 et 1) qui présentent respectivement la latence moyenne, le taux d'échec et l'utilisation CPU moyenne en fonction de la configuration et du niveau de charge.

Résultats principaux

Les résultats obtenus sur les 6 configurations testées montrent des tendances claires sur les trois métriques: ressources moyenne CPU consommées, durée moyenne d'une requête et taux d'échec moyen. Les configurations se comportent de façon similaire face à une faible charge (entre 1 et 10 requêtes). Des différences commencent à se voir à partir de 50 requêtes simultanées et s'accroissent beaucoup à partir de 500 requêtes.

Utilisation CPU. (Figure 1) À faible charge, toutes les configurations montrent une utilisation CPU proche de 0, sans différence significative. Une divergence apparaît ensuite, à partir de 50 requêtes: les configurations sans isolation (1 à 3) consomment entre 25% et 50% de CPU, alors que les configurations avec isolation (4 à 6) consomment entre 85% et 110% de CPU. Cette tendance s'accroît après 500 requêtes, où la configuration 6 (*granularité fine, isolation maximale*) dépasse 210% d'utilisation CPU en moyenne, avec des intervalles de confiance très larges traduisant une forte instabilité. Les configurations sans isolation se stabilisent autour de 45 à 50% quelle que soit la charge, cela montre une saturation de l'unique conteneur utilisé pour le déploiement.

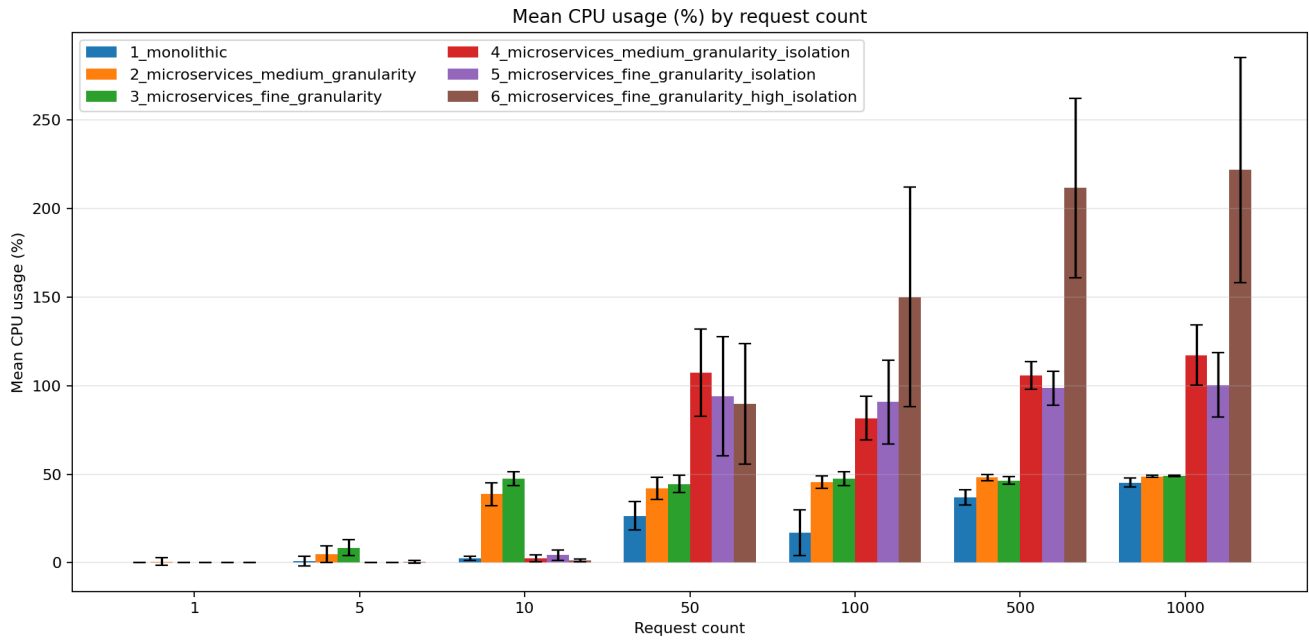


Figure 1. Utilisation CPU moyenne (en %) en fonction de la configuration et du niveau de charge

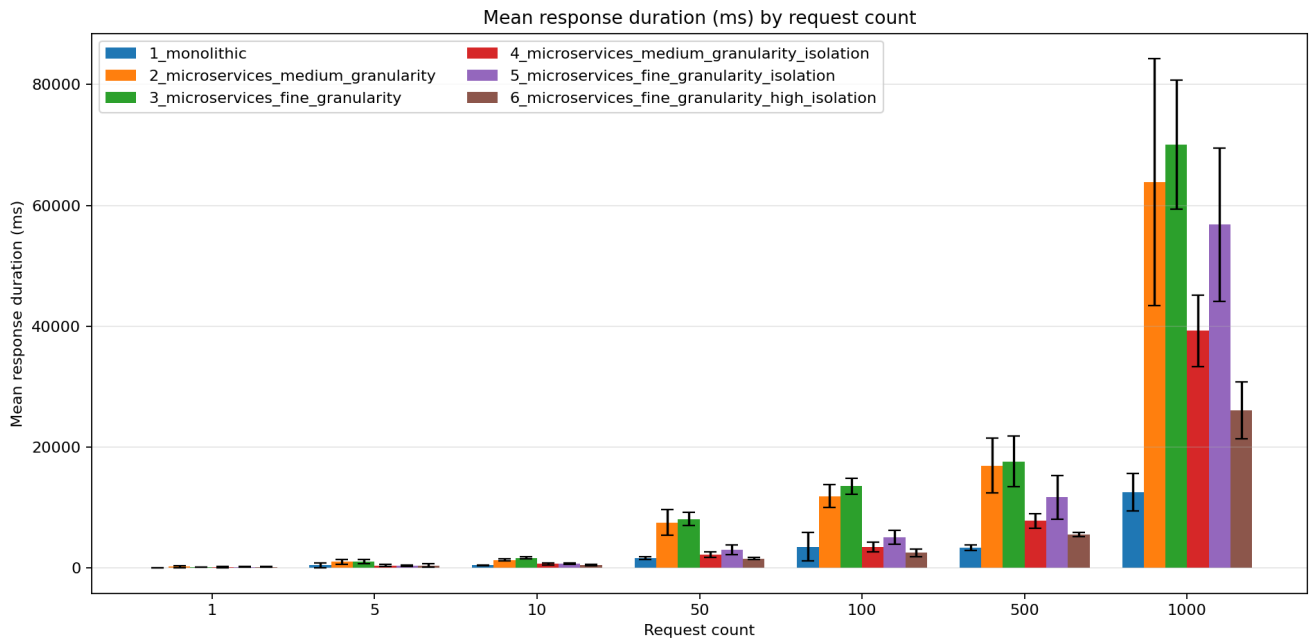


Figure 2. Latence moyenne (en ms) en fonction de la configuration et du niveau de charge

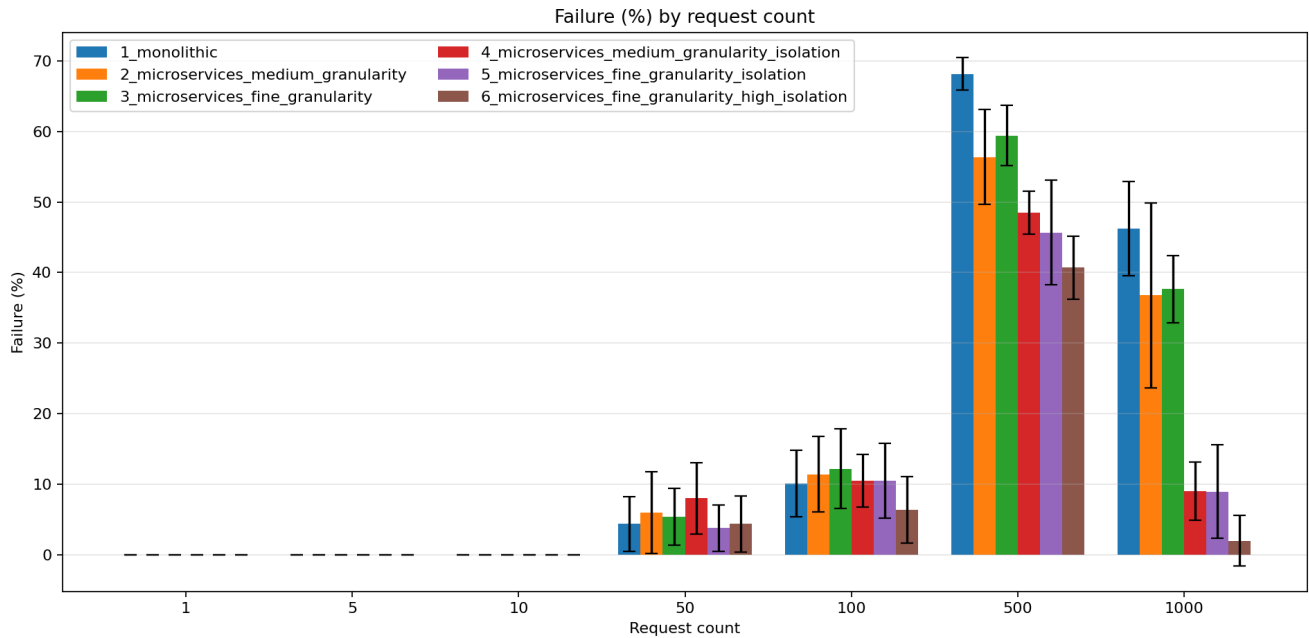


Figure 3. Taux d'échec en fonction de la configuration et du niveau de charge

Durée de réponse. (Figure 2) La durée de réponse moyenne ou latence moyenne reste sous 2000 ms pour toutes les configurations jusqu'à 100 requêtes en simultanées. Au-delà, les configurations sans isolation (2 et 3) montrent une croissance rapide de la latence qui atteint respectivement environ 65 000 ms et 70 000 ms à 1 000 requêtes. La configuration monolithique (1) affiche une latence intermédiaire d'environ 12 500 ms. À l'inverse, les configurations avec isolation conteneurisée (4 à 6) maintiennent des durées de réponse significativement inférieures à charge élevée. Ces résultats indiquent que l'isolation conteneurisée, bien qu'elle augmente la consommation CPU, permet de réduire efficacement la latence sous forte charge.

Taux d'échec. (Figure 3) On ne constate aucun échec pour les niveaux de charge allant de 1 à 10 requêtes. Les premiers apparaissent à 50 requêtes à des taux plutôt faible compris entre 4% et 8% si on considère que le service doit gérer au minimum 50 requêtes en même temps sur un seul serveur. À 500 requêtes, le taux d'échec atteint son maximum avec la configuration monolithique (1) qui culmine à environ 68%, suivie des autres entre 40 et 60% d'échec. À 1 000 requêtes, une inversion étrange de la tendance se distingue nettement avec une diminution du taux d'échec pour toutes les configurations. Ce comportement contre-intuitif s'explique par la saturation du système dès 500 requêtes, qui entraîne un rejet dès le début des requêtes en surcharge. On notera, cependant, que les configurations avec isolation ont tendance à mieux gérer la saturation et à limiter les échecs, avec la configuration 6 qui atteint un taux d'échec quasi nul à 1 000 requêtes (2%).

Analyse statistique

Les barres d'erreur représentées sur chaque graphique correspondent aux intervalles de confiance à 95% calculés sur les dix applications distribuées générées aléatoirement pour chaque configuration. Leur amplitude varie significativement selon la métrique et le niveau de charge considérés.

Pour l'utilisation CPU, les intervalles de confiance sont étroits à faible charge mais s'élargissent fortement à partir de 100 requêtes pour la configuration 6, indiquant une instabilité importante. Cela suggère que l'architecture de l'application devient un facteur important dans la consommation de ressources.

Pour la durée de réponse, les intervalles de confiance restent relativement faibles jusqu'à 1000 requêtes. Cela confirme l'instabilité relevée ci-dessus. On peut toutefois noter que celle-ci est moins représentée dans le cas d'une architecture avec isolation.

Résultats secondaires

Plusieurs observations secondaires peuvent être relevées.

Inversion du taux d'échec à 1 000 requêtes. Le comportement le plus inattendu concerne la diminution du taux d'échec entre 500 et 1 000 requêtes pour toutes les configurations. Ce phénomène suggère qu'au-delà d'un seuil de saturation, les architectures

rejetent activement les requêtes entrantes limitant ainsi le nombre de traitements tentés et donc d'échecs enregistrés. Ce mécanisme est très marqué pour la configuration 6, qui retrouve une quasi-absence d'échecs à 1 000 requêtes, au prix d'une consommation CPU très élevée.

Découplage entre consommation CPU et latence. Un des résultats les plus importants est l'absence de corrélation directe entre l'utilisation du CPU et la durée de réponse en fonction du mode de déploiement. Les configurations sans isolation 2 et 3 présentent des latences plus élevées à forte charge malgré une consommation CPU d'environ 45% tandis que la configuration 6 montre une latence plus faible à 1 000 requêtes malgré une utilisation du CPU dépassant 200%. Cette divergence s'explique par le fait que la configuration avec une forte isolation répartie la charge sur plus de processus parallèle, augmentant la consommation globale mais réduisant les temps d'attente par requête.

Stabilité du monolithique à charge modérée. La configuration monolithique (1) se trouve être compétitive jusqu'à 100 requêtes simultanées, affichant des performances comparables aux configurations microservices en termes de latence et de taux d'échec, pour une consommation CPU inférieure. Ce n'est qu'à partir de 500 requêtes que sa limitation architecturale apparaît avec un taux d'échec plus élevé que les autres configurations (68%). Cette observation confirme l'hypothèse H3 selon laquelle la concentration du traitement dans un service unique crée des surcharges lors d'une forte demande.

Discussion

Interprétation des résultats

Les résultats mettent en évidence trois mécanismes principaux.

À faible charge (1 à 10 requêtes), l'architecture n'a pas d'influence mesurable sur les performances: toutes les configurations se comportent de manière similaire, cela suggère que l'augmentation de la communication inter-services est négligeable lorsque l'application est peu sollicitée.

À charge moyenne (50 à 100 requêtes), deux logiques s'opposent. Dans la première, les configurations sans isolation (2 et 3) consomment peu de CPU grâce à des communications légères entre les services. Dans la seconde, la virtualisation a un coût important, mais le traitement en parallèle qu'elle apporte permet de garder la latence et le taux d'échec à des valeurs comparables au premier cas.

Enfin, à forte charge (500 à 1 000 requêtes), le facteur limitant bascule: ce n'est plus le surcoût en communication qui détermine les performances, mais la capacité du système à gérer les requêtes concurrentes. Les configurations sans isolation saturent rapidement car elles ne peuvent paralléliser le traitement sur plusieurs processus indépendants. Le monolithe (1) est caractéristique de ce phénomène avec le taux d'échec le plus élevé à 500 requêtes (68%), confirmant l'hypothèse H3. À l'inverse, la dernière configuration (6) tire parti de l'isolation maximale pour maintenir la latence la plus basse au prix d'une forte utilisation du CPU (plus de 200%).

Comparaison avec la littérature

Les résultats obtenus s'inscrivent globalement dans la continuité des travaux existants, tout en apportant des nuances.

La tendance observée (granularité fine sans isolation entraîne les latences les plus élevées à forte charge) est en accord avec les conclusions de Ricardo César Mendonça Filho et Nabor C. Mendonça⁵, qui montrent que le découplage des services augmente significativement la surcharge de communication entre processus. De plus, comme le soulignent les observations de Yiming Zhao, Tiziano De Matteis et Justus Bogner⁴, l'impact de la granularité augmente avec la taille de l'application, ce qui rejoint nos observations de l'instabilité des performances à mesure que la charge augmente.

À l'inverse, deux résultats s'éloignent des conclusions de la littérature. Le premier concerne l'impact de la conteneurisation sur la latence. Alors que Dharmendra Shadija, Mo Rezai et Richard Hill² concluent à une augmentation négligeable, notre étude nuance ce constat: si l'écart est faible à faible charge, il devient non négligeable à partir de 500 requêtes. Cela suggère que la visibilité du surcoût dépend de la charge du système. Le second résultat concerne la séparation entre consommation CPU et latence. Contrairement à l'intuition selon laquelle une consommation CPU élevée traduit une baisse des performances, les configurations à isolation maximale combinent ici la plus forte utilisation CPU et la latence la plus faible à haute charge. Ce découplage n'a pas été abordé dans les études de référence et il constitue une conclusion non attendue.

Implications pratiques

Ces résultats permettent de formuler trois recommandations à destination des architectes logiciels.

Privilégier l'isolation conteneurisée dès que la charge anticipée dépasse un seuil critique. En dessous de 50 requêtes simultanées, le mode de déploiement importe peu. Au-delà, l'isolation devient un levier efficace pour contenir la latence et le taux d'échec, au prix d'une consommation CPU plus élevée.

Éviter l'architecture monolithique pour les systèmes à fort trafic. Malgré des performances compétitives à faible charge, le monolithe présente le taux d'échec le plus élevé à 500 requêtes. Sa simplicité ne compense pas son incapacité à répondre à toutes les requêtes sous forte charge.

Adapter la granularité à l'objectif prioritaire. Une granularité fine avec isolation maximale (configuration 6) offre la plus faible latence à haute charge, mais mobilise des ressources CPU importantes et présente une variabilité élevée selon la topologie de l'application (il faudra donc faire des tests par architecture). Une granularité moyenne avec isolation (configuration 4) offre un meilleur compromis, combinant une plus faible consommation CPU et des performances satisfaisantes.

Limites et recherches futures

Limites méthodologiques

Cette étude repose sur une simulation contrôlée qui simplifie certains aspects des systèmes distribués réels. Les microservices simulés implémentent des tâches génériques sans modéliser des comportements complexes tels que des accès concurrents à des ressources ou des appels à des services externes. De plus, nous n'avons pu générer que 10 architectures avec 12 microservices chacune, ce qui ne couvre pas l'espace complet des architectures rencontrées en production (cela a pris une dizaine d'heure pour toutes les tester). Enfin, l'étude ne se penche que sur 3 métriques alors que les facteurs importants en production peuvent être tout autre.

Validité et généralisabilité

La validité interne de l'expérience est assurée par le contrôle strict des variables: même matériel, mêmes scripts de génération de charge, mêmes conditions d'exécution pour toutes les configurations. Les résultats sont donc comparables entre eux au sein du protocole défini. En revanche, la validité externe est limitée: les conclusions s'appliquent majoritairement à des systèmes distribués de taille moyenne, dans un environnement sans contraintes réseau artificielles ni variabilité d'infrastructure. Leur comparaison à des environnements cloud multi-régions ou à des systèmes comportant des centaines de services doit être faite avec prudence.

Pistes de recherches futures

Trois axes se dégagent. En premier, une validation sur des systèmes réels permettrait de confronter les résultats de simulation aux comportements émergents des orchestrateurs de conteneurs (comme Kubernetes). En second lieu, l'intégration de la consommation énergétique comme métrique principale permettrait une analyse coût-performance plus complète, en lien avec les enjeux de sobriété numérique. Enfin, l'introduction d'injections de pannes contrôlées dans le protocole expérimental permettrait d'évaluer en même temps performance et résilience (capacité d'un système à se remettre d'une panne).

Conclusion

Synthèse des principaux enseignements

Cette étude a évalué l'influence de la granularité des microservices et du déploiement conteneurisé sur les performances d'une application distribuée simulée, à travers six configurations croisées testées sur dix applications et sept niveaux de charge. Les résultats montrent que le mode de déploiement est le facteur le plus important à forte charge: l'isolation conteneurisée permet de contenir la latence et le taux d'échec au prix d'une consommation CPU élevée, tandis que les architectures sans isolation saturent rapidement. Le monolithique, compétitif à charge modérée, présente le taux d'échec le plus élevé au-delà de 500 requêtes.

Recommandations

Pour les applications à charge faible ou prévisible, une architecture monolithique ou à granularité moyenne sans isolation reste pertinente et moins coûteuse en ressources. Pour les systèmes soumis à des pics de charge importants, une granularité moyenne avec isolation conteneurisée (configuration 4) constitue le meilleur compromis entre performance, prévisibilité et consommation CPU. La granularité fine avec isolation maximale (configuration 6) est à réserver aux systèmes dont la priorité absolue est la limitation des échecs, à condition de disposer des ressources matérielles nécessaires pour absorber la surcharge CPU induite.

Perspectives

Au-delà du cadre académique de cette étude, les compromis identifiés entre granularité, isolation et charge constituent des leviers concrets pour les équipes d'architecture logicielle confrontées à la conception d'une architecture logicielle. L'émergence d'outils comme Service Weaver permettant de modifier le niveau de granularité sans changer le code applicatif ouvre la voie à une optimisation dynamique de l'architecture en fonction des conditions d'exploitation observées. La question de la granularité optimale ne se pose alors plus uniquement au moment de la conception, mais tout au long du cycle de vie du système.

References

1. C. Medeiros Rêgo, R. C. Mendonça Filho, and N. C. Mendonça, “Performance and Resilience Impact of Microservice Granularity: An Empirical Evaluation Using Service Weaver and Amazon EKS,” *International Journal of Network Management*, vol. 35, no. 4, p. e70019, 2025.
2. D. Shadija, M. Rezai, and R. Hill, “Microservices: Granularity vs. Performance,” in *Companion Proceedings of The 10th International Conference on Utility and Cloud Computing*. Austin Texas USA: ACM, Dec. 2017, pp. 215–220.
3. F. H. Vera-Rivera, C. Gaona, and H. Astudillo, “Defining and measuring microservice granularity—a literature overview,” *PeerJ Computer Science*, vol. 7, p. e695, Sep. 2021.
4. Y. Zhao, T. D. Matteis, and J. Bogner, “How Does Microservice Granularity Impact Energy Consumption and Performance? A Controlled Experiment,” in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*, Mar. 2025, pp. 84–95.
5. R. C. Mendonça Filho and N. C. Mendonça, “Performance Impact of Microservice Granularity Decisions: An Empirical Evaluation Using the Service Weaver Framework,” in *Software Architecture*, M. Galster, P. Scandurra, T. Mikkonen, P. Oliveira Antonino, E. Y. Nakagawa, and E. Navarro, Eds. Cham: Springer Nature Switzerland, 2024, vol. 14889, pp. 191–206.

Annexes

Le code source du projet et l'ensemble des résultats sont normalement fournis avec ce document ou sur le [GitHub](#).